

# Dictionary in Python

- Python dictionary is an unordered, changeable and indexed collection of items. Each item of a dictionary has a key/value pair. An empty dictionary without any items is written with just two curly braces, like this: {}.
- Keys are unique within a dictionary while values may not be. The values of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.

## Creating Python Dictionary

In [37]:

```
d1 = {}
d2 = {1: 'apple', 2: 'ball'}
d3={'name': 'John', 1: [2, 4, 3]}
d4=dict({1:'apple', 2:'ball'})
d5=dict([(1,'apple'), (2,'ball')])
d6=dict(name='John', age=2)
print(d1)
print(d2)
print(d3)
print(d4)
print(d5)
print(d6)
```

```
{}
```

```
{1: 'apple', 2: 'ball'}
```

```
{'name': 'John', 1: [2, 4, 3]}
```

```
{1: 'apple', 2: 'ball'}
```

```
{1: 'apple', 2: 'ball'}
```

```
{'name': 'John', 'age': 2}
```

## Accessing Elements from Dictionary

- While indexing is used with other data types to access values, a dictionary uses keys. Keys can be used either inside square brackets [] or with the get() method.
- If we use the square brackets [], KeyError is raised in case a key is not found in the dictionary. On the other hand, the get() method returns None if the key is not found.

In [2]:

```
d3={'name': 'John', 1: [2, 4, 3]}
print(d3['name'])
#print(d3['age']) raise error as key is not there in dictionary
print(d3.get('name'))
print(d3.get('age'))
```

```
John
John
None
```

## Altering and adding element in dictionary

In [3]:

```
d1={'name': 'Antra', 'age': 26}
d1['age']=24
print(d1['age'])
d1['address']="Anand"
print(d1)
```

24

```
{'name': 'Antra', 'age': 24, 'address': 'Anand'}
```

## Removing elements from Dictionary

- We can remove a particular item in a dictionary by using the pop() method. This method removes an item with the provided key and returns the value.
- The popitem() method can be used to remove and return an arbitrary (key, value) item pair from the dictionary. All the items can be removed at once, using the clear() method.
- We can also use the del keyword to remove individual items or the entire dictionary itself.

In [19]:

```
squares = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36}
print(squares.pop(4))
print(squares.popitem())
print(squares.popitem())
print(squares)
squares.clear()
print(squares)
```

16

(6, 36)

(5, 25)

{1: 1, 2: 4, 3: 9}

{}

In [20]:

```
del squares
#print(squares) will raise error as dictionary is not existing after del
```

## Python Dictionary Methods

In [27]:

```
squares = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36}
print(squares.items()) #Return a new object of the dictionary's items in (key, value) format
print(squares.keys()) #Returns a new object of the dictionary's keys.
print(squares.values()) #Returns a new object of the dictionary's values
d = {1: "one", 2: "three"}
d1 = {2: "two"}
d.update(d1) #updates the dictionary with elements from other dictionary
print(d)
d2 = {1: "one", 2: "three"}
d3=d2.copy()
print(d3)
```

```
dict_items([(1, 1), (2, 4), (3, 9), (4, 16), (5, 25), (6, 36)])
dict_keys([1, 2, 3, 4, 5, 6])
dict_values([1, 4, 9, 16, 25, 36])
{1: 'one', 2: 'two'}
{1: 'one', 2: 'three'}
```

## Python Dictionary Comprehension

- Dictionary comprehension is an elegant and concise way to create a new dictionary from an iterable in Python.
- Dictionary comprehension consists of an expression pair (key: value) followed by a for statement inside curly braces {}.

In [28]:

```
squares = {x: x*x for x in range(6)}
print(squares)
```

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

In [29]:

```
odd_squares = {x: x*x for x in range(11) if x % 2 == 1}
print(odd_squares)
```

```
{1: 1, 3: 9, 5: 25, 7: 49, 9: 81}
```

## Other Dictionary Operations

In [30]:

```
squares = {1: 1, 3: 9, 5: 25, 7: 49, 9: 81}
print(1 in squares)
```

```
True
```

## Iterating Through a Dictionary

In [31]:

```
# Iterating through a Dictionary
squares = {1: 1, 3: 9, 5: 25, 7: 49, 9: 81}
for i in squares:
    print(squares[i])
```

```
1
9
25
49
81
```

## Dictionary Built-in Functions

In [1]:

```
squares = {0: 0, 3: 9, 5: 25, 7: 49, 9: 81, 1: 1}
print(all(squares)) #Return True if all keys of the dictionary are True (or if the dictionary is not empty)
print(any(squares)) #Return True if any key of the dictionary is true. If the dictionary is not empty, it returns True.
print(len(squares))
print(sorted(squares)) #Return a new sorted list of keys in the dictionary.
```

```
False
True
6
[0, 1, 3, 5, 7, 9]
```

In [6]:

```
#Write a program to map 2 lists into a dictionary.
my_dic= {}
list1= [1, 2, 3, 4]
list2= ['blue', 'red', 'green', 'black']
for i in range (0, len(list1)):
    my_dic.update({list1[i]:list2[i]})
my_dic
```

Out[6]:

```
{1: 'blue', 2: 'red', 3: 'green', 4: 'black'}
```